

UNIT: 6

Structure, Union and Files

❖ Structure

❖ Introduction to Structures and Declaration, Initialization and accessing of Structures:

- Structure is a collection of logically related data items with different data type.
- Structure must be defined first, that may be used later to declare structure.

Syntax:

```
struct structure_name
{
    datatype member1;
    datatype member2;
    ---
    datatype memberN;
};
```

- struct is a keyword, it declares structure to hold data.
- Structure_Name is the name of structure (structure tag).
- member1, member2,... are members of structure.

Example:

```
struct student
{
    char name[20];
    int roll_no;
    float per;
}s1;
```

Here **s1** is a structure variable.

Structure Initialization:

Example:

```
struct student
{
    char name[20];
    int roll_no;
    float per;
}s1={"Arav",20,85.5};
```

Example:

```
#include<stdio.h>
struct student
```

```
{
    char name[20];
    int roll_no;
    float per;
}s1;
void main()
{
    struct student s1={"Arav",20,85.5};

    printf("\nName      : %s",s1.name);
    printf("\nRollNo     : %d",s1.roll_no);
    printf("\nPercent    : %f",s1.per);
}
```

❖ Introduction to union and Declaration, Initialization and accessing of union:

- Union is a group of memory that is used to store variables of different datatypes.
- Union are same as structure, but main difference between structure and union is in **terms of storage**.

Syntax :

```
union union-name
{
    datatype member1;
    datatype member2;
    ---
    datatype memberN;
};
```

Example:

```
union student
{
    char name[20];
    int roll_no;
    float per;
};
```

Example:

```
union student
{
    char name[20];
    int roll_no;
    float per;
}u1;
```

```

void main()
{
    u1.roll_no=65;
    printf("\nRollNo      : %d",u1.roll_no);
}

```

Difference between structure and union

	Structure	union
1	In structure, each member has separate memory allocated.	In union all member shares common memory.
2.	Any member can be accessed at any time.	Only one member can be accessed at a time.
3.	<pre> struct data { int a; float b; char c; }; </pre>	<pre> union data { int a; float b; char c; }; </pre>
4.	Size of structure is total memory allocated to each member. Size of student= total sizeof(a,b,c);	Size of union is maximum memory allocated to any member. Size of student= maximum memory from (a,b,c);
5	Structure is defined using struct keyword.	Union is defined using union keyword.

Difference between Array and structure.

	Array	Structure
1	An array is a collection of variables of same data type known by same name.	A structure is a collection of variables of different data type known by same name.
2.	An array is a derived data type.	A structure is a programmer defined.
3.	For array we have to declare an array variable and use it.	For structure we have to design and declare a data structure before the variables of that type are declared and used.

4.	Example: int a[5];	Example: struct data { int a; float b; char c; };
----	-----------------------	---

❖ User-defined Data types: enum, typedef

Enumerated Data Type:

- Enumerated data type is user defined data type.
- Using enumerated data type we can create more than one symbolic constant at a time.

Syntax: **enum identifier {value1, value2,value3};**

Here, enum is a keyword to declare enumerated data type.
Identifier is a user defined enumerated data type.

For Example:

```
#include<stdio.h>
enum week {Mon=1, Tue, Wed, Thur, Fri, Sat, Sun};
void main()
{
    enum week day;
    day = Wed;
    printf("%d",day);
    getch();
}
```

Output:2

typedef :

The typedef is a keyword that is used in C programming to provide existing data types with a new name. typedef keyword is used to redefine the name already the existing name.

When names of datatypes become difficult to use in programs, typedef is used with user-defined datatypes.

Syntax:

typedef <existing_name> <alias_name>

Example:

typedef long mylong

❖ Files

Introduction to text Files

File:

A file is a group of related data which is stored in a disk.

File operations:

- (1) Naming a file
- (2) Opening a file
- (3) Reading data from a file
- (4) Writing data to a file
- (5) Closing a file

Basic file operations:

fopen() - create a new file or open a existing file

fclose() - close a file

getc() - reads a character from a file

putc() - writes a character to a file

fscanf() - reads a set of data from a file

fprintf() - writes a set of data to a file

getw() - reads a integer from a file

putw() - writes a integer to a file

fseek() - set the position to desire point

ftell() - gives current position in the file

rewind() - set the position to the beginning point

fopen() and fclose():

fopen(): Create a new file or open an existing file.

Syntax:

```
FILE *fp;
```

```
fp=fopen("filename", "mode");
```

Here,fp is a file pointer to the datatype FILE.

Second statement open a file named filename and mode specify the opening of this file.

Example:

```
FILE *fp;
```

```
fp=fopen("E:\\abc.txt", "r");
```

fclose(): Close the file

A file must be closed as soon as all operation on it have been completed.

Syntax:

```
fclose(file pointer);
```

Example:

```
FILE *fp;
```

```
fp=fopen("E:\\abc.txt", "r");
```

```
-----  
fclose();
```

fgetc() and fputc():**fputc():** Write character to the file

Syntax: fputc(c,fp);

Here c is a character type variable and fp is a file pointer.

This statement write the character stored in variable c to the file.

Example:

```
void main()  
{  
char c='A';  
FILE *fp;  
Fp= fopen(" E:\\ abc.txt", "w");  
fputc(c,fp);  
fclose(fp);  
getch();  
}
```

fgetc(): Read character from file

Syntax: c=fgetc(fp);

Here c is a character type variable and fp is a file pointer.

Example:

```
void main()  
{  
char c;  
FILE *fp;  
Fp= fopen(" E:\\ abc.txt", "r");  
c = fgetc(fp);  
printf("%c", c);  
fclose(fp);  
getch();  
}
```

fscanf() and fprintf():

These functions are used for reading and writing the data to the file.

fprintf() : Write all data written in list to the file.

Syntax: fprintf(fp , "Control String ", List);

Where fp is a file pointer.

List may include variable, constants and string.

Example:

```
void main()  
{
```

```
char name[10]="abc";
int id=1;
FILE *fp;
Fp= fopen(" E:\\ abc.txt", "w");
    fprintf(fp,"%s %d", name,id);
fclose(fp);
getch();
}
```

fscanf() : Read all data written in file and store in variables.

Syntax: fscanf(fp , "Control String ", List);

Where fp is a file pointer.

List may include variable, constants and string.

Example:

```
void main()
{
char name[10];
int id;
FILE *fp;
Fp= fopen(" E:\\ abc.txt", "r");
    fscanf(fp,"%s %d", name, &id);
    printf("Name : %s and id : %d", name, id);
fclose(fp);
getch();
}
```

Example of reading data from a file and writing to it(in text mode):

```
#include<stdio.h>
#include<conio.h>
void main()
{
FILE *fp;
char ch;
fp = fopen("hello.txt", "w");
printf("Enter data");
while( (ch = getchar()) != EOF) {
    putc(ch,fp);
}
fclose(fp);
fp = fopen("hello.txt", "r");

while( (ch = getc(fp)) != EOF)
    printf("%c",ch);
}
```

```
fclose(fp);  
}
```


UNIT: 5

Functions

❖ Introduction to Functions:

Definition of Function: Function is a group of statement to perform a specific task.

Types of functions:

1. Library function
2. User-defined Function

❖ Advantages of using Functions:

1. By using functions, we can avoid rewriting same logic/code again and again in a program.
2. We can call C functions any number of times in a program and from any place in a program.
3. We can track a large C program easily when it is divided into multiple functions.

❖ Types of Functions: Built-in and user defined Functions

Library function(Built-in):

- The function which is develop by system itself is known as Library function .
- Example: printf(),scanf(),sqrt(),ceil() etc ...
- Library function is all ready exist in system.

User-defined Function:

- The function which is develop by user itself is known as User-defined function.
- Example: main()
- User-defined function is created by the user at the time of writing of program.

Difference between Library function and User-defined Function.

No.	Library function	User-defined Function
1	The function which is develop by system itself is known as Library function.	The function which is develop by user itself is known as User-defined function.
2.	Example: printf(),scanf(),sqrt() etc ...	Example: main(),add()
3.	There no elements in Library function.	User-defined function has three elements.

❖ Declaring, Defining and calling user defined Functions

(1) Function declaration

Syntax:

ReturnType FunctionName (Parameter List);

- Function Type is a return type or datatype of a function.
- Function name is the name of the user defined function.

Example:

```
int sum (int, int);  
int sum( int a, int b);
```

(2) Function call

Syntax:

Identifier = Function name(Argument value);

Where identifier is the name of variable

Example:

```
a = sum(10,5);
```

(3) Function definition

Syntax:

```
ReturnType Function Name(Parameter List)  
{  
    Local variable declaration  
    Function statement  
    Return Statement  
}
```

Example:

```
int sum(int x, int y)  
{  
    int total;  
    total = x + y;  
    return total;  
}
```

❖ Categories of user-defined Functions

A function may or may not accept any argument. It may or may not return any value. Based on these facts, There are four different categories of function calls.

- function without arguments and without return value
- function without arguments and with return value
- function with arguments and without return value

- function with arguments and with return value

Example for Function without argument and return value

```
#include<stdio.h>
void sum();
void main()
{
    printf("\n sum of two numbers:");
    sum();
}
void sum()
{
    int a,b;
    printf("\nEnter two numbers");
    scanf("%d %d",&a,&b);
    printf("The sum is %d",a+b);
}
```

Example for Function without argument and with return value

```
#include<stdio.h>
int sum();
void main()
{
    int result;
    result = sum();
    printf("%d",result);
}
int sum()
{
    int a,b;
    printf("\nEnter two numbers");
    scanf("%d %d",&a,&b);
    return a+b;
}
```

Example for Function with argument and without return value

```
#include<stdio.h>
void sum(int, int);
void main()
{
    int a,b,result;
    printf("\nEnter two numbers:");
    scanf("%d %d",&a,&b);
    sum(a,b);
}
void sum(int a, int b)
{
    printf("\nThe sum is %d",a+b);
}
```

Example for Function with argument and with return value

```
#include<stdio.h>
int sum(int, int);
void main()
{
    int a,b,result;
    printf("\nEnter two numbers:");
    scanf("%d %d",&a,&b);
    result = sum(a,b);
    printf("\nThe sum is : %d",result);
}
int sum(int a, int b)
{
    return a+b;
}
```

❖ Call by Value and call by Reference**Call By Value with example:**

- When a function is called using the value of variables, then it is known as call by value.

- The value of variables, which are to be passed, will be copied from the variables of the calling functions to the variables of the called functions.
- All the process done on the duplicate variables rather than actual variables.

Example:

```
#include <stdio.h>
void swap(int , int);    // function prototype
int main()
{
    int a = 10;
    int b = 20;
    printf("Before swapping the values in main a = %d, b = %d\n",a,b);
    swap(a,b);    // function call
    printf("After swapping values in main a = %d, b = %d\n",a,b); //actual parameters a=10, b=20
}
}
void swap (int a, int b)
{
    int temp;
    temp = a;
    a=b;
    b=temp;
    printf("After swapping values in function a = %d, b = %d\n",a,b); // Formal parameters, a=20, b=10
}
```

Call By Reference with example:

- When a function is called using the address of variables, then it is known as call by reference.
- Instead of passing the value of variables from calling function to the called function, addresses of the variables are passed.
- All the process done on the actual variables.

Example:

```
#include <stdio.h>
void swap(int *, int *);    // function prototype
void main()
{
    int a = 10;
    int b = 20;
    printf("Before swapping the values in main a = %d, b = %d\n",a,b);
```

```
    swap(&a,&b);        // function call
    printf("After swapping values in main a = %d, b = %d\n",a,b); // actual parameters a = 20, b = 10
}
void swap (int *a, int *b)
{
    int temp;
    temp = *a;
    *a=*b;
    *b=temp;
    printf("After swapping values in function a = %d, b = %d\n",*a,*b); // Formal parameters a=20, b=10
}
```

❖ Recursion

- Function call itself is called Recursion.
- Recursive call to a function may be conditional or unconditional.
- In conditional recursive call, function is terminated when condition is false.
- In unconditional recursive, function is called infinitely.
- It is used to find the factorial of a given number.
- Example :

```
void main()
{
    printf("Recursion");
    main();
}
```

- Example :

```
void main()
{
    -----
    function1();
    -----
}
function1()
{
    -----
    function1();
    -----
}
```

Advantages of Recursion:

- The recursion is very flexible in data structure like stacks.
- Using recursion, the length of the program can be reduced.

Disadvantages of Recursion:

- It requires extra storage space.
- The recursion function is not efficient in execution speed and time.
- Proper termination is required; otherwise it leads to infinite loop.

❖ Built-in Functions: String and Maths**Built-in Functions:****C Maths Function:**

The <math.h> header file contains various methods for performing mathematical operations such as sqrt(), pow(), ceil(), floor() etc.

ceil(number)

Rounds up the given number.

It returns the integer value which is greater than or equal to given number.

Ex: ceil(3.6)

It returns 4.

floor(number)

Rounds down the given number. It returns the integer value which is less than or equal to given number.

Ex: floor(3.6)

It returns 3.

sqrt(number)

Returns the square root of given number.

Ex: sqrt(16)

It returns 4.

pow(base, exponent)

Returns the power of given number.

Ex pow(2,4)

It returns 16.

abs(number)

Returns the absolute value of given number.

Ex: abs(-16)

It returns 16.

C String Functions:

There are many important string functions defined in "string.h" library.

No.	Function	Meaning
1)	strlen(string_name)	returns the length of string name.
2)	strcpy(destination, source)	copies the contents of source string to destination string.
3)	strcat(first_string, second_string)	concatenates or joins first string with second string. The result of the string is stored in first string.
4)	strcmp(first_string, second_string)	compares the first string with second string. If both strings are same, it returns 0.
5)	strrev(string)	returns reverse string.
6)	strlwr(string)	returns string characters in lowercase.
7)	strupr(string)	returns string characters in uppercase.

❖ Storage Classes: -auto, static, register and extern**Storage Classes in C:**

Storage classes in C are used to determine the lifetime, visibility, memory location, and initial value of a variable. There are four types of storage classes in C

- Automatic
- External
- Static
- Register

Storage Classes	Storage Place	Default Value	Scope	Lifetime
auto	RAM	Garbage Value	Local	Within function
extern	RAM	Zero	Global	Till the end of the main program Maybe declared anywhere in the program
static	RAM	Zero	Local	Till the end of the main program, Retains value between multiple functions call
register	Register	Garbage Value	Local	Within the function

UNIT: 4

Array and Pointers

❖ Introduction of Array:

Definition: Array is a collection of variables of same data type known by same name.

Types of Arrays:

- One-dimensional arrays.
- Two-dimensional arrays.
- Multidimensional arrays.

❖ Characteristics of an array:

- Array store elements that have same data type.
- Array store elements in subsequent memory location.
- Array size should be mention in the declaration.
- Array name represent the address of starting elements.

❖ Declaration and initialization of 1- D (One dimensional array):

Declaration of array:

Syntax:

`datatype array_name[size];`

Data type: it defines the datatype, like int,float,char,double etc.

Array name: it is the name of array.

Size: It represents the size of the array.

Example:

```
int a[5];  
float f[5];
```

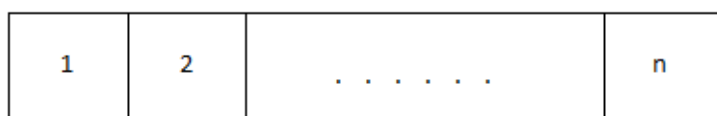


Fig: One Dimensional Array

Initialization of array:

(1) Compile time:

`datatype arrayname[size]={List of value};`

`int a[5]={10,20,30,40,50};`

(2) Run time:

```
for(i=0 ; i<5 ; i++)  
{  
    scanf("%d", &a[i]);  
}
```

Example: Program to read 5 elements of array and print it.

```
#include<stdio.h>  
#include<conio.h>  
void main()  
{  
    int a[5],i;  
    clrscr();  
    printf("Enter 5 elements of array");  
    for(i=0;i<5;i++)  
    {        scanf("%d",&a[i]);  
    }  
    printf("The array elements are:\n");  
  
    for(i=0;i<5;i++)  
    {  
        printf("%d\n",a[i]);  
    }  
    getch();  
}
```

❖ Declaration and initialization of 2- D (Two dimensional array):**Declaration of array:**Syntax:

datatype Arrayname [row size] [column size];

Example:

```
int a[3][3];
```

where, int is datatype, a is a arrayname , row size is 3 and column size is 3.

```
float f[5][10];
```

Initialization of array:**(1) Compile time:**

```
int a[3][3]={ {11,12,13}, {14,15,16}, {17,18,19} } ;
```

```
int a[3][3] = {11,12,13,14,15,16,17,18,19};
```

	col 0	col 1	col 2
row 0	11	12	13

	a[0][0]	a[0][1]	a[0][2]
row 1	14 a[1][0]	15 a[1][1]	16 a[1][2]
row 2	17 a[2][0]	18 a[2][1]	19 a[2][2]

(In memory Representation of array)**(2) Run time:**

```
for(i=0 ; i<3 ; i++)  
{  
    for(j=0 ; j<3 ; j++)  
    {  
        scanf(" %d ", &a[i][j] );  
    }  
}
```

Example: Program to read 3X3 matrix (2D array elements) and print it.

```
#include<stdio.h>  
void main()  
{  
    int i,j,a[3][3];  
    printf("Enter elements of 3X3 matrix\n");  
    for(i=0;i<3;i++)  
    {  
        for(j=0;j<3;j++)  
        {  
            scanf("%d",&a[i][j]);  
        }  
    }  
    printf("The elements of 3X3 matrix\n");  
    for(i=0;i<3;i++)  
    {  
        for(j=0;j<3;j++)  
        {  
            printf("%d\t",a[i][j]);  
        }  
    }  
    printf( "\n" );  
}
```

❖ Introduction to a String:

String:

A string is a sequence of zero or more characters followed by a NULL '\0' character.

String is always terminating with NULL '\0' character.

Declaration and initialization of string:

Syntax:

```
char stringname[size];
```

where size is the number of characters.

Example:

```
char city[10];
```

Initialization of string variable:

(1) Compile time:

```
char city[9]= " NEW YORK";
```

(2) Run time:

```
char city[9];
```

```
scanf("%s", city);
```

OR

```
char city[9];
```

```
gets(ciy);
```

❖ gets() and puts() :

gets():

This function read a string from a user. It is defined in the <stdio.h> header file of C.

puts() :

This function prints a string on the console screen. It is also defined in the <stdio.h> header file of C.

Example: Program to read and print a string using gets() and puts().

```
void main()
{
char day[10];
printf("Enter day: \n");
gets(day);
```

```
printf("Today is: ");
puts(day);
}
```

Output:

Enter day: TUESDAY

Today is: TUESDAY

❖ Pointer

❖ Introduction to Pointer:

Pointer is a variable which store the address of another variable.

Advantage of pointer:

- Pointer reduces the code and improves the performance.
- We can return multiple values from a function using the pointer.
- It makes you able to access any memory location in the computer's memory.

Characteristics of Pointers:

- Pointer is a variable which can hold the address of another variable.
- A pointer is a derived data type.
- It contains memory addresses as their values.
- If a C pointer is assigned to the null value, it points nothing.
- The asterisk symbol, * is used to retrieve the value of the variable
- & ampersand symbol is used for retrieving the address of a variable.

❖ declaration and initialization of pointer :

Declaration of pointer

Syntax:

Data type *pointer-name

Example:

```
int *p;
```

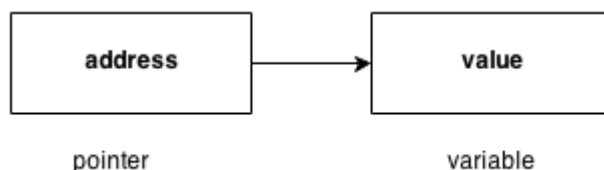
Initialization of pointer:

Syntax:

Pointer variable= &variable-name;

Example:

```
int price;  
int *p;  
p=&price;
```



Example:

```
#include<stdio.h>
Void main(){
int number=50;
int *p;
p=&number;//stores the address of number variable
printf("Address of p variable is %x \n",p); // p contains the address of the number
printf("Value of p variable is %d \n",*p); // * is used to dereference a pointer therefore *p means get the
value stored at the address contained by p.
}
```

❖ Types of pointer : void and null

void pointer

Syntax:

void *variable name

- Void pointer is a pointer which has no specified data type.
- It is also known as generic pointer.
- Void pointer can be pointed to any data type.
- When void pointer is declared two bytes of memory is assign to it.
- It is used when return type or parameter is unknown.
- void pointers that have addresses, can be further typecast into other data types very easily.

Example:

```
include<stdio.h>
void main()
{
int a = 10;
void *p;
p = &a;
printf("%d", *(int *)p);
}
```

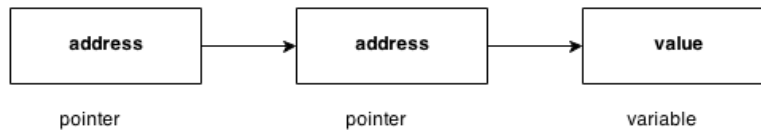
NULL Pointer:

- A pointer that is not assigned any value but NULL is known as the NULL pointer.
- If you don't have any address to be specified in the pointer at the time of declaration, you can assign NULL value. It will provide a better approach.

int *p=NULL;

pointer to pointer:

- Pointer variable may point to another pointer variable.
- It is also known as double pointer.
- The first pointer is used to store the address of a variable whereas the second pointer is used to store the address of the first pointer.



Syntax:

Data type **pointer variable name;

Example:

```
int **p; // pointer to a pointer which is pointing to an integer.
```

Example:

```
int n,*p1,**p2;
n=10;
p1=&n;
p2=&p1;
```

Example:

```
#include<stdio.h>
void main ()
{
    int a = 10;
    int *p;
    int **p2;
    p = &a; // pointer p is pointing to the address of a
    p2 = &p; // pointer p2 is a double pointer pointing to the address of pointer p
    printf("address of a: %x\n",p); // Address of a will be printed
    printf("address of p: %x\n",p2); // Address of p will be printed
    printf("value stored at p: %d\n",*p); // value stored at the address contained by p i.e. 10 will be
    printed
    printf("value stored at p2: %d\n",**pp); // value stored at the address contained by pointer
    stored at p2
}
```


UNIT: 3

Decision Statements and Control Structure

- ❖ Decision Statements
- ❖ Conditional branching statements:

Simple if statement:

Syntax:

```
if (condition)
{
    Statement (s)
}
```

If the given condition is true then the statements inside the body of “if” will be executed. If the condition is false then the statements inside the body of “if” will be skipped.

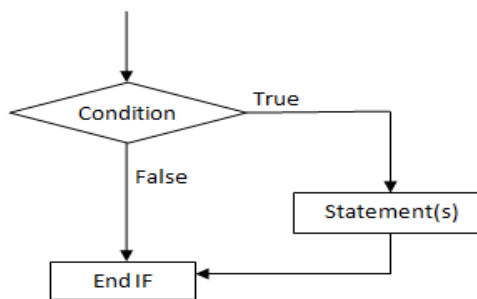


fig: Flowchart for if statement

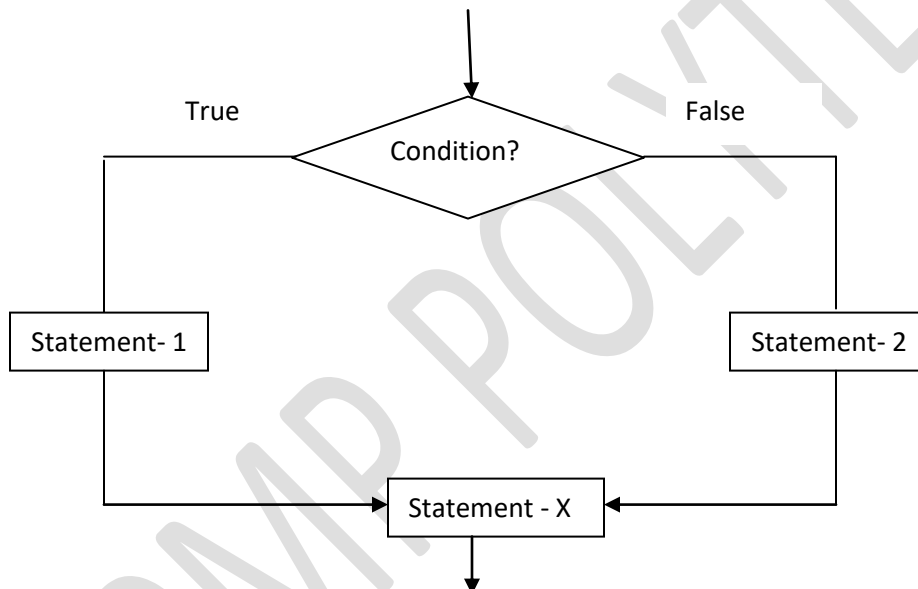
```
#include<stdio.h>
#include<conio.h>
void main()
{
    int marks=45;
    if( marks < 35 )
    {
        printf("Student is fail");
    }
    getch();
}
```

If-else statement:Syntax:

```
if(condition)
{
    Statement 1;
}
else
{
    Statement 2;
}
```

The if..else statement is executed in the following order:

1. first the condition is checked.
2. if condition is true the statement -1 is executed.
3. if condition is false the statement - 2 is executed.

**Example:**

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int marks=45;
    if( marks < 35 )
    {
        printf("Student is fail");
    }
}
```

```
}  
else  
{  
printf( "Student is pass" );  
}  
getch();  
}
```

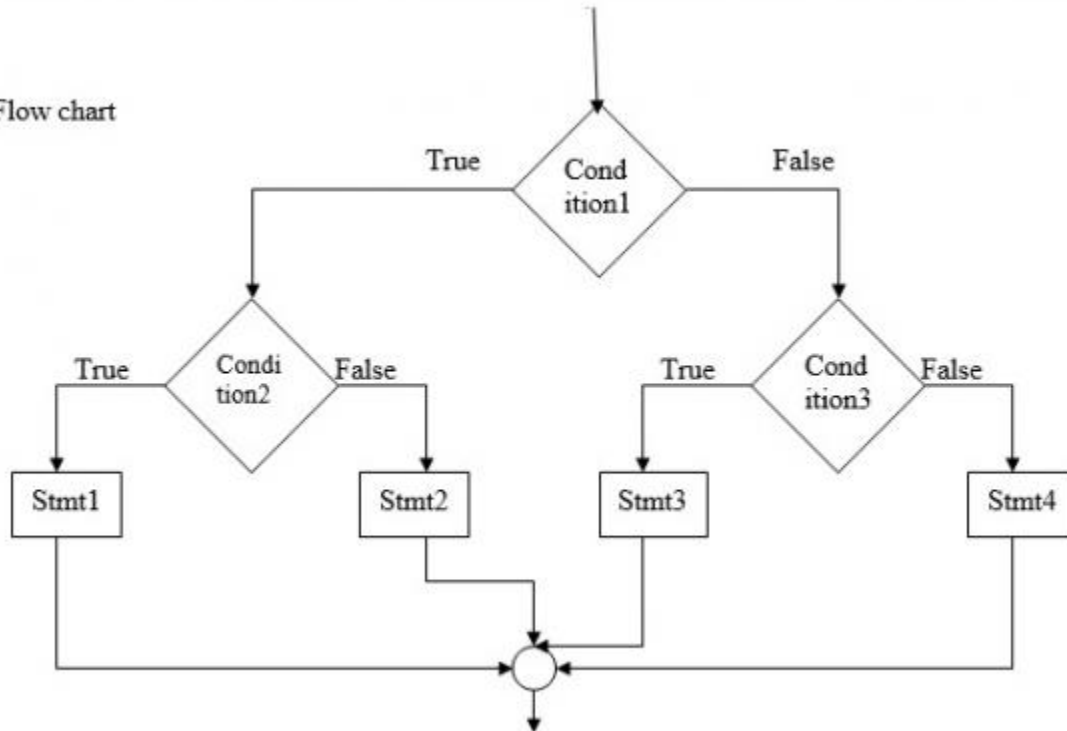
Nested If-else statement:

Nested If-else statement means if-else statement within another if-else statement.

Syntax:

```
if (condition1)  
{  
    if (condition2)  
        stmt1;  
    else  
        stmt2;  
}  
else  
{  
    if (condition3)  
        stmt3;  
    else  
        stmt4;  
}
```

Flow chart



Example:

```
#include<stdio.h>
#include<conio.h>
void main (){
    int a,b,c;
    printf("Enter the values of a,b,c: ");
    scanf("%d,%d,%d",&a,&b,&c);
    if((a>b)&&(a>c))
    {
        if(a>c)
        {
            printf("%d is the largest",a);
        }
    }
    else
    {
        printf("%d is the largest",c);
    }
}
else
{
    if(b>c)
    {
        printf("%d is the largest",b);
    }
}
```

```
else
{
    printf("%d is the largest",c);
}
```

if else if...ladder

It is used **for multiple choice**.

Syntax:

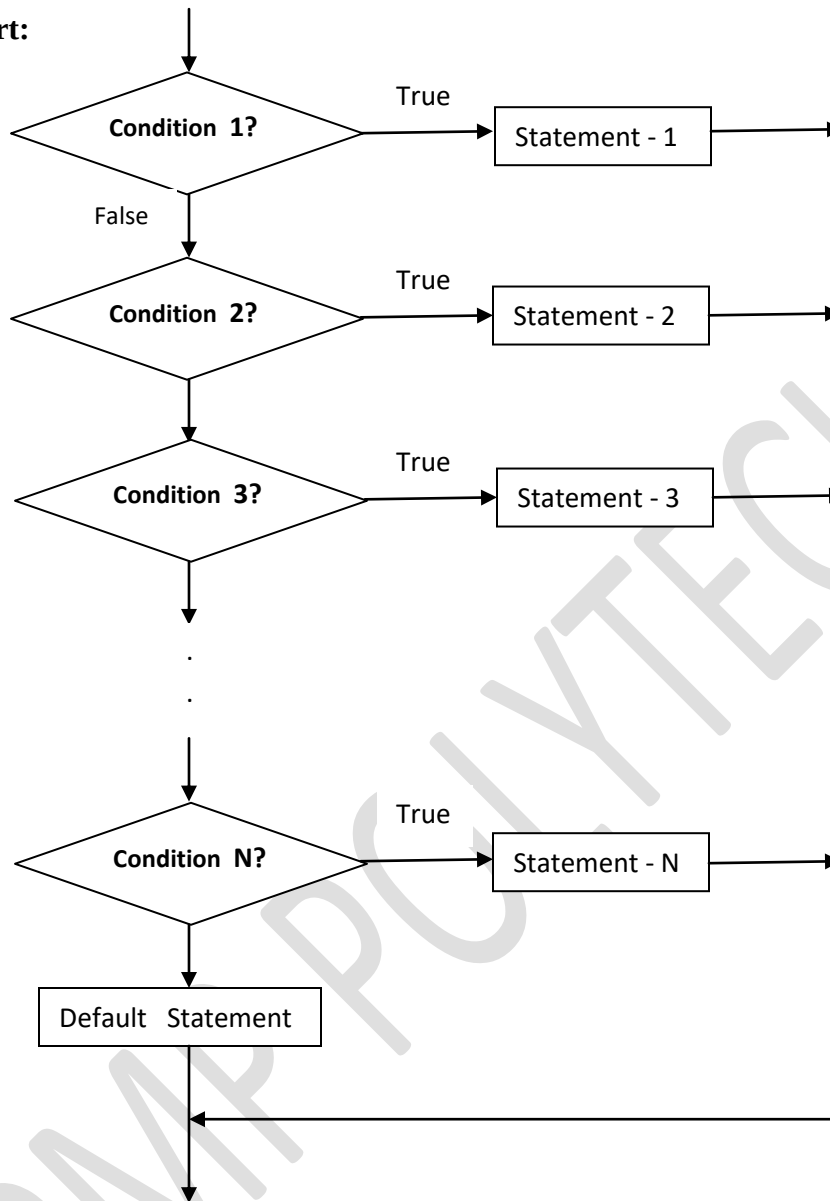
```
if(condition-1)
{
    Statement - 1;
}
else if(condition-2)
{
    Statement - 2;
}
else if(condition-3)
{
    Statement - 3;
}
.
.
.
else if(condition-N)
{
    Statement -N;
}
else
{
    Default statement;
}
```

if-else-if ladder is executed in the following order:

1. First condition-1 is executed, if the condition-1 is true then the statement-1 is executed.
2. if the condition-1 is false then condition-2 is checked. If condition-2 is true then statement-2 is executed.

3. This procedure repeated until all the condition is checked. if all the condition became false then the default statement is executed.

Flowchart:



Example:

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int no;
    printf("enter number\n");
    scanf("%d",&no);
    if( no > 0 )
    {
        printf("Number is Positive");
    }
    else if (no < 0)
    {
        printf( "Number is Negative" );
    }
    else
    {
        printf( "Number is Zero" );
    }
    getch();
}
```

Switch statement:

- It is used to select one among multiple decisions.
- The switch statement is also known as multi-choice or multi decision statement.
- 'switch' successively tests a value against a list of integers or character constant.
- When a match is found, the statement(s) associated with that value is executed.
- Break statement send the control to the next statement after switch statement.
- If the case not found then the statement associated with the default case is executed.

Syntax:

```
switch(variable name or expression)
{
case label1:
    statement(s) 1;
    break;
case label2:
    statement(s) 2;
    break;
    .
    .
case label N:
    statement(s) N;
    break;
default :
    default statement(s);
}
```

Example:

```
void main()
{
int a=2;
clrscr();

switch(a)
{

case 1:
    printf("One \n");
    break;
case 2:
    printf("Two");
    break;
case 3:
    printf("Three");
    break;
default:
    printf("Wrong choice");

getch();
}
```

O/p: Two

❖ Unconditional branching statement: goto

- The go to statement in the c programming is used to transfer the control unconditionally from one part of the program to other part of the program.
- 'c' language provide a **unconditional branching mechanism** called as go to statement.

Syntax:

```
goto label ;
```

- Here , label is the label to the statement to which go to transfer control.
- Label must be a valid identifier.
- There are two possible use of goto statement.
 1. **Forward Reference**
 2. **Backward Reference**

Forward reference	Backward reference
goto label; Label: Statement;	Label: Statement; Goto label;
Target statement comes after the goto.	Target statement comes before the goto.

❖ Control Statements:

When we want to repeatedly run a particular statement until a particular condition is met, then in that case we use Loop Statements.

There are three types of loop statements in C language:

- While loop
- Do while loop
- For loop

While loop:

- It is entry control loop.
- In the while loop, first the condition is checked and if the condition is true then the statement inside the while loop is run.
- As soon as the condition is False, the control is moved out of the while loop, and another statement is executed.
- At first time, condition is false, then body is not executed.

Syntax:

```
while (Condition)
{
    Statements;
}
```

Example:

```
i =1;

while (i <= 10)
{
    printf( "% d \n " , i );
    i = i+1;
}
```

Do-While loop:

- It is exit control loop.
- In do-while loop, first body is executed and then condition is checked and if the condition is true, then again body of the loop will be executed, otherwise next statement after the loop will be executed.
- At first time, condition is false, at least once the body is executed.

Syntax:

```
do
{
    // statements inside the loop
}
While (condition);
```

Example:

```
i =1;
do
{
```

```
printf( "% d \n " , i );  
i = i+1;  
} while (i <= 10);
```

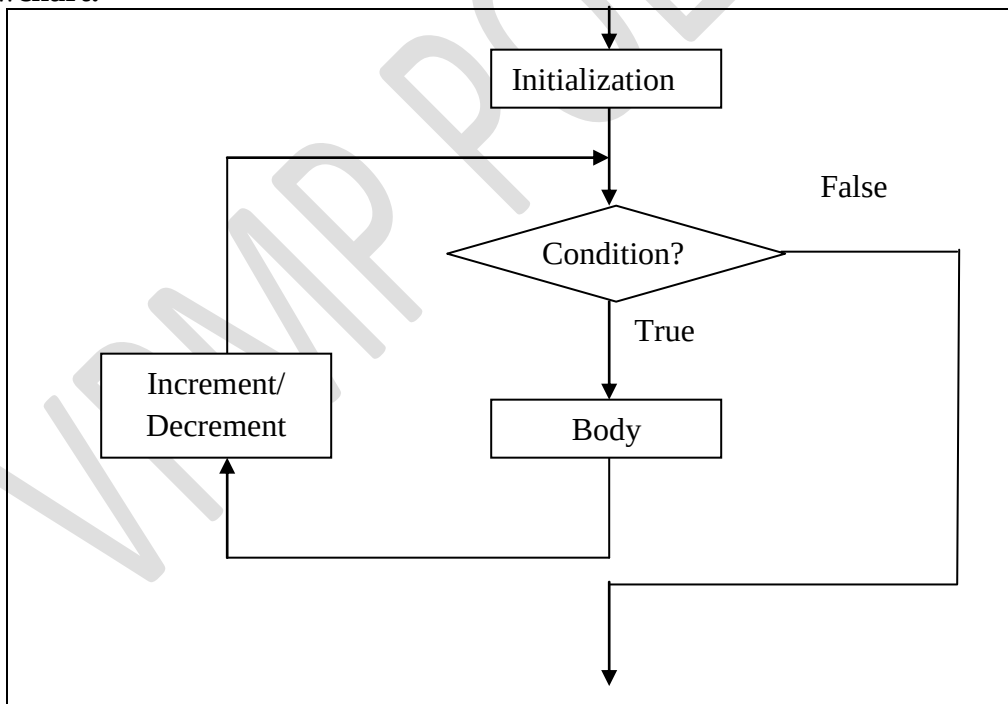
For loop :

- In For Loop, we write both the initialization and control condition of the variable together inside the parentheses “()”.
- If the condition of For Loop is true, then the statement inside it is run, or else the statement is not run.

Syntax:

```
for (Initialization ; Condition ; Increment or decrement)  
{  
    body of the loop;  
}
```

- **Initialization:** The control variable is initialized in the initialization expression. It executes only once.
- **Condition:** The condition statement checks the value of the control variable. If the condition statement is true, the body of the loop is executed.
- **Increment/Decrement:** The increment/decrement of the control variable is done in this part. After the incrementing/decrementing the value of control variable, it is tested using condition if condition is true then again the body of loop is executed and this process is repeated until the condition become false.

Flowchart:

Example: Print 1 to 5 numbers using for loop.

```
for(i = 1; i <=5 ; i++)
{
    printf(" %d \n ", i ) ;
}
```

Nested for loop:

Nested for loop means for loop within another for loop.

Syntax:

```
for (Initialization; condition; increment ) {

    for (Initialization; condition; increment ) {
        statement(s);
    }
    statement(s);
}
```

Example:

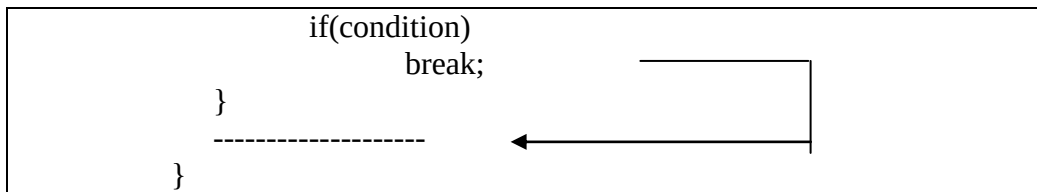
```
for(i=1;i<=5;i++)
{
    for(j=1;j<=i;j++)
    {
        printf("%d",i);
    }
    printf("\n");
}
```

❖ Break and continue statements:

Break statement:

- It terminates the loop or switch statement and transfers execution to the statement immediately following the loop or switch.
- Keyword used is "**break**".
- Break from loop:

```
for(      ;      ;      )
{
    for(      ;      ;      )
    {
```



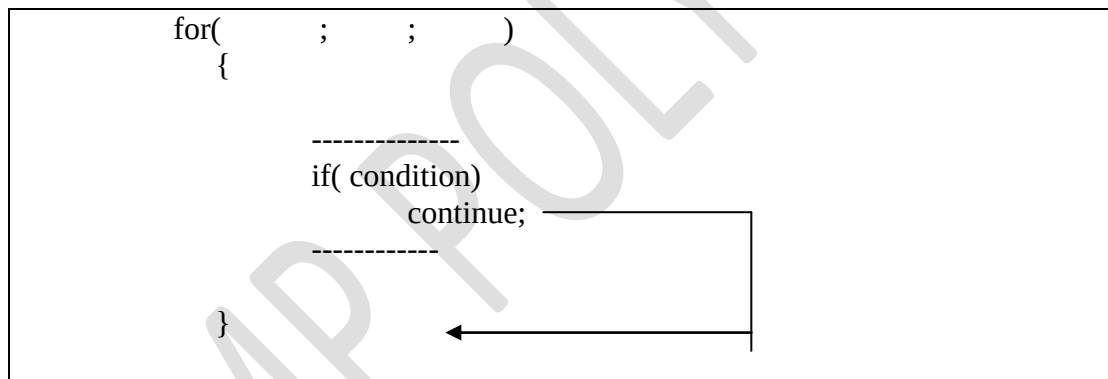
Example;

```
for(i=1; i<=5 ; i++)
{
    if(i==3)
        break;
    printf("%d\n",i);
}
```

Output: 1 2

Continue statements :

- It causes the loop to skip the remainder of its body and starts the next iteration.
- Keyword used is "**continue**".



Example:

Print 1 to 5 number except 3.

```
for( i=1 ; i <= 5 ; i++)
{
    if ( i ==3)
        continue;
    printf("%d ",i);
}
```

Output: 1 2 4 5

UNIT: 2

Basics of 'C'

❖ General structure of 'C' program:

Documentation section
Header File
Constants and Global variables Declaration
main () function section
{
Statement(s);
}
User Defined functions and procedures with their body

Fig:structure of 'C' program

Documentation section:

- Documentation consists of a set of comment line(`/* */`) giving the name of the program.

Header File(Link Section):

- Link section provides instruction to the compiler to link function from the system library(Header Files like `#include<stdio.h>`).

Constant Declaration:

- Definition section defines all symbolic constants
Ex: `#define PI 3.14`

Global declaration section:

- There are some variable that are used more than one function. Such Variables are called global variable and that is outside all the function.

main () function section:

- Every c program must have one main () function section.
- This section contains two parts. Declaration part and executable part.
- Declaration part declares all the variables used in the executable part.
- These two parts must be appearing between opening and closing braces.

User Defined functions and procedures:

- The subprogram section contains all the user defines functions that are called in the main function.

Features of C language:

1) C is portable.

‘C’ Program can be run on any hardware.

2) C supports variety of built-in functions.(library functions printf() and scanf())

3) C supports bitwise operators.

The C support bitwise operator like AND, OR, NOT etc. normally these operations do not supported by higher level languages.

4) C is modular language.

The program written in from of functions. A C program is group of one or more functions. Dividing the program into small functions makes it to easy develop and maintain the programs.

5) C is Robust language.

❖ Character set:

In the C programming language, the character set refers to a set of all the valid characters that we can use in the source program for forming words, expressions, and numbers.

There are four type of character set in C programming.

- Alphabets - Upper case A-Z , Lower case a-z
- Digit- 0-9
- Special Character - %, &, *, etc.
- White space

❖ ‘C’ tokens, keyword and identifiers

- **C Token** : The smallest individual unit in a C program is known as C token.

C has six types of token as given below:

- Keywords (float, while)
- Identifier (main, amount)
- Constants (-15.2, 10)
- String (“abc”, “hello”)
- Special Symbols ([,] , * , ! etc)
- Operators (+, -, *, /)

- **Keyword:**

Keywords are those words whose meaning is predefined.

- The programmer cannot change the meaning of keywords.
- There are 32 keywords available in c.
- All the keywords must be written in lowercase.
- Examples:

int if float while

- **Identifiers:**

- Identifier is the words which are defined by the programmer in a program.
- Identifier refers to the name of variable, array, function etc.
- It consists of letters, digits and underscore.
- First character must be an alphabet or underscore.
- Keywords cannot be use as identifier.
- Identifier name are case sensitive.
- The name of identifier should be meaningful.

❖ Data Types in 'C':

Primary (basic) data type with size and range:

Data type is used to specify which type of value to be stored in variable.

Type	Size(Bytes)	Size(Bits)	Range
char	1	8	-128 to +128
int	2	16	-32768 to +32767
float	4	32	3.4 e -38 to 3.4 e +38
double	8	64	1.7 e -308 to 1.7 e +308

❖ Variable

A **variable** is a name which is used to store a temporary value.

Rules of variables Naming:

- The variable name may consist of letters (A-Z, a-z), digits (0-9) and underscore symbol (_).
Example: int no_1; int no1; int NO1;
- The variable name must begin with a letter. Some system permits underscore as a first character.
Example: int 1NO; is not valid
- The length should not be more than eight characters.
- Variable name should not be a keyword.

Example: int case is not valid

- A white space is not allowed in the name of variable.

Example: int NO 1; is not valid.

- Variable is case sensitive. Uppercase and lowercase are different.

Example: int MAX; int max;

Here both MAX and max are treated as different variable.

❖ **constant variable:**

- Constant means fixed value.
- If the value of a variable remain constant during the execution of a program, then we can declare the variable with **const** at the time of initialization.

Syntax: const DataType ConstantName

Example: const float PI = 3.14;

❖ **Introduction of different types of operators :**

Operators in C

- 1) Arithmetic operator
- 2) Relational operator
- 3) Logical operator
- 4) Assignment operator
- 5) Increment and decrement operator
- 6) Conditional (ternary) operator
- 7) Bitwise operator
- 8) Special operator

Arithmetic operator :

- +, -, *, /, % are arithmetic operators.
- +, -, *, / are used with any data type.
- % is used only with integer type.

Arithmetic Operators:

1) Binary plus(+):

- ✓ It adds two operands.
- ✓ Example :a+b

2) Binary minus(-):

- ✓ It subtracts two operands.
- ✓ Example :a-b

3) Multiplication(*):

- ✓ It multiplies two operands

- ✓ Example :a*b

4) Division(/) :

- ✓ It divides one operand by another operand.

- ✓ Example: a/b

5) Modulo operator (%):

- ✓ It divides one operand by another operand and produce remainder of the division.

- ✓ Example :a%b

Now assume that a=5 and b=2 then

1) a+b =7

2) a-b =3

3) a*b =10

4) a/b =2

5) a%b =1

Relational operator:

Relational operators are used for comparisons.

i) Equal to(=)

It compares two operands and return TRUE (1) if both operands are equal.

ii) Not equal to (!=)

It compares two operands and return TRUE (1) if both operands are not equal.

iii) Less than(<)

It compares two operands and return TRUE (1) if operand1 is less than operand2.

iv) Less than equal to(<=)

It compares two operands and return TRUE (1) if operand1 is less than or equal to operand2.

v) Greater than(>)

It compares two operands and return TRUE (1) if operand1 is greater than operand2.

vi) Greater than equal to(>=)

It compares two operands and return TRUE (1) if operand1 is greater than or equal to operand2.

Examples:

Suppose a=5, b=7

1) a= b false

2) a!=b true

3) a<b true

4) a<=b true

5) a>b false

6) a>=b false

Assignment Operator:

Operator	Description	Example
=	Simple assignment operator. Assigns values from right side operands to left side operand	$C = A + B$ will assign the value of $A + B$ to C
+=	Add AND assignment operator. It adds the right operand to the left operand and assign the result to the left operand.	$C += A$ is equivalent to $C = C + A$
-=	Subtract AND assignment operator. It subtracts the right operand from the left operand and assigns the result to the left operand.	$C -= A$ is equivalent to $C = C - A$
*=	Multiply AND assignment operator. It multiplies the right operand with the left operand and assigns the result to the left operand.	$C *= A$ is equivalent to $C = C * A$
/=	Divide AND assignment operator. It divides the left operand with the right operand and assigns the result to the left operand.	$C /= A$ is equivalent to $C = C / A$
%=	Modulus AND assignment operator. It takes modulus using two operands and assigns the result to the left operand.	$C \% = A$ is equivalent to $C = C \% A$

Logical operator:

- Logical operators are used when we want to combine more than one condition.

Logical AND	&&
Logical OR	
Logical NOT	!

Logical AND(&&)

- The truth table logical AND (&&) operator is given below:

Condition 1	Condition 2	Answer
False	False	False
False	True	False

True	False	False
True	True	True

Logical OR(||)

- The truth table logical OR (||) operator is given below:

Condition 1	Condition 2	Answer
False	False	False
False	True	True
True	False	True
True	True	True

Logical NOT (!):

- The truth table logical NOT (!) operator is given below:

Condition	Answer
False	True
True	False

- Examples :**

Suppose a=5, b=3, c=6;

- 1) (a>b) && (a>c) - false
- 2) (a>b) || (a>c) - true
- 3) !(a>b) - false

Increment and decrement operator:

Increment Operators are the unary operators used to increment or add 1 to the operand value.

The Increment operand is denoted by the double plus symbol (++).

It has two types, Pre Increment and Post Increment Operators.

Pre-increment Operator:

The pre-increment operator is used to increase the original value of the operand by 1 before assigning it to the expression.

X = ++A;

In the above example, the value of operand 'A' is increased by 1, and then a new value is assigned to the variable 'X'.

Post increment Operator:

The post-increment operator is used to increment the original value of the operand by 1 after assigning it to the expression.

$X = A++;$

In the above example, the value of operand 'A' is assigned to the variable 'X'. After that, the value of variable 'A' is incremented by 1.

Pre- decrement Operator:

The pre- decrement operator is used to decrease the value of the operand by 1 before assigning it to the expression.

$X = --A;$

In the above example, the value of operand 'A' is decreased by 1, and then a new value is assigned to the variable 'X'.

Post decrement Operator:

The post- decrement operator is used to decrement the original value of the operand by 1 after assigning it to the expression.

$X = A--;$

In the above example, the value of operand 'A' is assigned to the variable 'X'. After that, the value of variable 'A' is decremented by 1.

Bitwise operator:

- The bitwise operators are used to manipulate data at bit level.
- These operators are used **for testing bits** or **shifting them left or right**.
 - 1) Bitwise AND(&):
 - 2) Bitwise OR(|):
 - 3) Bitwise Exclusive OR(^):
 - 4) Shift left(<<):
 - 5) Shift right(>>):
 - 6) One 's complement(~):

Bit 1	Bit 2	Bit1 & Bit2	Bit1 Bit2	Bit1 ^ Bit2
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Example: Bitwise AND(&)

$x=5$
 $y=7$
 $z = x \& y$

$$x \rightarrow 0000\ 0101$$
$$y \rightarrow 0000\ 0111$$
$$z \rightarrow 0000\ 0101$$

Example: Bitwise or(|)

$$x=5, \quad y=7$$
$$z = x \& y$$
$$x \rightarrow 0000\ 0101$$
$$y \rightarrow 0000\ 0111$$
$$z \rightarrow 0000\ 0111$$

Example: Bitwise Exclusive OR(^)

$$x=5, \quad y=7$$
$$z = x \& y$$
$$x \rightarrow 0000\ 0101$$
$$y \rightarrow 0000\ 0111$$
$$z \rightarrow 0000\ 0010$$

Conditional operator (ternary operator):

- The conditional operator is also known as ternary operator because it has three operands.
- The general form of ternary operator is follow:
 - **Syntax:** $e1 ? e2 : e3 ;$
 - where $e1$ is logical expression(condition) , and evaluated first.
- If the condition is true then the statement is followed the? Is executed($e2$ is executed) otherwise the statement followed the : is executed($e3$ is executed).

Example:

$$\text{max} = (x > y) ? x : y;$$

First $(x > y)$ is checked, if it is true then x is assigned to max , otherwise y is assigned to max .

❖ Operator precedence and their associativity:

Operator precedence determines the grouping of terms in an expression and decides how an expression is evaluated.

Category	Operator	Associativity
Postfix	() [] -> . ++ --	Left to right
Unary	+ - ! ~ ++ -- (type)* & sizeof	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Left to right
Shift	<< >>	Left to right
Relational	< <= > >=	Left to right
Equality	== !=	Left to right
Bitwise AND	&	Left to right
Bitwise XOR	^	Left to right
Bitwise OR		Left to right
Logical AND	&&	Left to right
Logical OR		Left to right
Conditional	?:	Right to left
Assignment	= += -= *= /= %= >>= <<= &= ^= =	Right to left
Comma	,	Left to right

Evaluation of Expressions:

- The expressions are evaluated by performing one operation at a time. The precedence and associativity of operators decide the order of the evaluation of individual operations.
- The expressions with parentheses are evaluated first. If two or more parentheses exist in an expression, the parentheses are evaluated from left to right. In the case of nested parentheses, the innermost parentheses are evaluated first, while the outermost one is evaluated last.
- If there are no parentheses, the order of evaluation of an expression is based on the operator precedence and associativity:

Example:

```

7 * (5 + 15) / (2 * 5) - 3
= 7*20/10-3.
= 140/10-3
=14-3
=11

```

❖ Type casting /Type conversion :

- The type conversion are automatic as part of expression evaluation.
- But some times the user needs explicit type conversion.
- Example, int / int is always int. But, if we need the answer in float , then type casting is used.

Syntax:

Example: (type _ name) expression ;

```
float average;  
int sum=25, n=10;  
average = (float) sum / n;
```

where sum / n results into 2.5 rather than 2, because the division is performed after converting sum in to float and the result of float / int is always float. So,

```
average =(float) sum / n;  
= (float) 25 / 10;  
= 25.0 / 10  
= 2.5
```

❖ Input and Output statements in 'C':

- When we say Input, it means to feed some data into a program. An input can be given in the form of a file or from the command line. C programming provides a set of built-in functions to read the given input and feed it to the program as per requirement.
- When we say Output, it means to display some data on screen, printer, or in any file. C programming provides a set of built-in functions to output the data on the computer screen as well as to save it in text or binary files.

Formatted input and output in 'C':

C provides standard functions scanf() and printf(), to perform formatted inputs and outputs.

scanf() function:

Syntax:

```
scanf(format_specifiers, &data1, &data2,.....);
```

Example:

```
scanf("%d %c",&data1,&data2);
```

printf() function:

Syntax:

```
printf(format_specifiers, data1, data2,.....);
```

Example:

```
printf("%d %c",data1,data2);
```


UNIT: 1

Flowchart and Algorithm

❖ Flowchart

Definition and Importance of flowchart:

Flowchart: Flowchart is Graphical or Pictorial Representation of problem.

❖ Importance (Advantages) of Flowchart:

- 1.Flowcharts are better way of communicating the logic of the system.
- 2.Using Flowchart it is easy to analyze Problem.
- 3.The flowcharts are very useful during program development phase.
- 4.Flowchart represents problem solution using different standard Symbol.
- 5.It provides a good communication interface between programmer and user.
- 6.It helps in debugging and testing process.

❖ Symbols of flowchart:

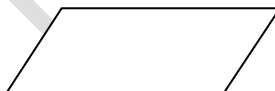
1.Terminal:

- This Symbol is used for the Starting and Ending Process of Flowchart.



2.Input / Output

- This Symbol is used for Receiving input Data From user and displaying output to the user



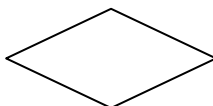
3.Process

- This Symbol is used for Computing.
- It is Use for Some Arithmetic Calculation.



4.Decision Making

- This Symbol is used for making decision base on Condition.
- This Symbol has one incoming arrow for entry and Two outgoing arrow.

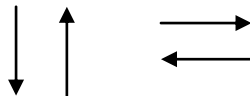


5.Subroutine

- This Symbol is used for user defined function or sub program.

**6.Flow / Direction**

- Flow line Symbol.
- This Symbol Indicates flow of data from one Symbol to Another Symbol.

**7.Page Break**

- When flowchart is do not fit into single page then Page Break and Page Continue Symbol is used.
- Page Break Symbol Indicate break in current page.

**8.Page Continue**

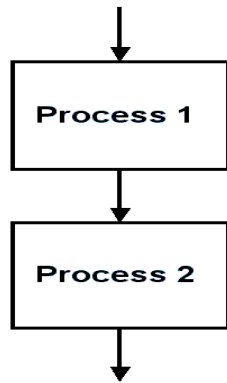
- Page Continue Symbol Indicate Continue flowchart in another page.

**❖ Guidelines for preparing Flowchart:**

1. Standard symbols should be used while drawing flowchart.
2. Ensure that flowchart has START (or BEGIN) and STOP (or END).
3. Flowchart should be neat, clean and easy to follow. There should be no any ambiguity.
4. The usual direction of flowchart is from top to bottom or from left to right.
5. The terminal symbol, that is, START/BEGIN or STOP/END should have only one flow line.
6. Only one flow line should come out from process symbol.
7. Only one flow line should enter a decision symbol.

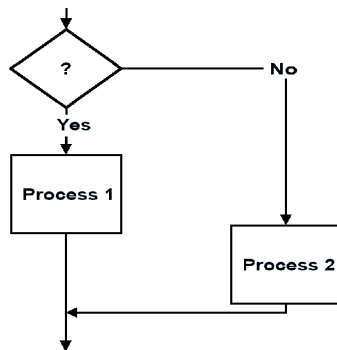
❖ Flowchart structure: Sequence, selection, repetition:**1. Sequence**

Steps that execute in sequence are represented by symbols that follow each other top to bottom or left to right. Top to bottom is the standard.



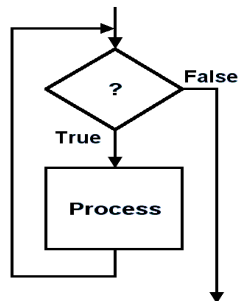
2. Selection

Once the condition is evaluated, the control flows into one of two paths.



3. Repetition

Either the processing repeats or the control leaves the structure.



❖ Limitation of flowchart:

1. Complex Logic: It is very difficult to draw a flowchart for very hard and complex problem
2. Difficulty in Modifications: If a minor change is required then it is difficult to change flowchart and may be required to redraw flowchart again.
3. Difficult to understand for people who don't know flowchart symbols.

❖ Definition and Importance of Algorithm:

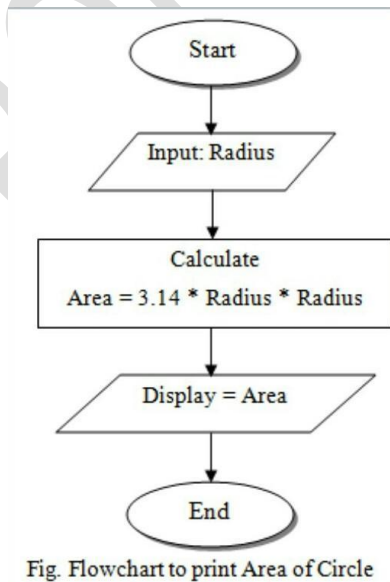
Algorithm: Algorithm is stepwise solution of any problem or program.

Importance (advantages) of Algorithm:

1. It is a step-wise representation of a solution to a given problem, which makes it easy to understand.
2. An algorithm uses a definite procedure.
3. It is not dependent on any programming language, so it is easy to understand for anyone even without programming knowledge.
4. Every step in an algorithm has its own logical sequence so it is easy to debug.
5. By using algorithm, the problem is broken down into smaller pieces or steps, so it is easier for programmer to convert it into an actual program.

Limitation of Algorithm

1. An algorithm is Time consuming. Developing algorithm for complex problems would be time consuming and difficult to understand.
2. Understanding complex logic through algorithms can be very difficult.
3. Difficult to show Branching and Looping in Algorithms.
4. Big tasks are difficult to put in Algorithms.

Examples:**Flowchart and algorithm to calculate area of circle.****Flowchart:**

Algorithm:

1. START
2. INPUT: r
3. Calculate: $\text{Area} = 3.14 * r * r$
4. PRINT: Area
5. STOP

❖ Developing and writing algorithm using pseudo codes:**Pseudo code:**

Pseudo code is not a programming language; it is a simple way of describing a set of instructions that does not have to use specific syntax.

Example:

If student's grade is greater than or equal to 60

Print "passed"

else

Print "failed"